



PASSION *for* TECHNOLOGY

16.05.2026 Czapiewski Paweł
POJ Ćwiczenia 05

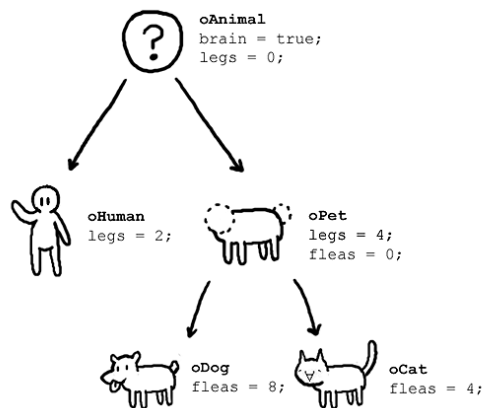


Source: <https://www.religie.424.pl/smierc-w-egipcie,4004,article.html>

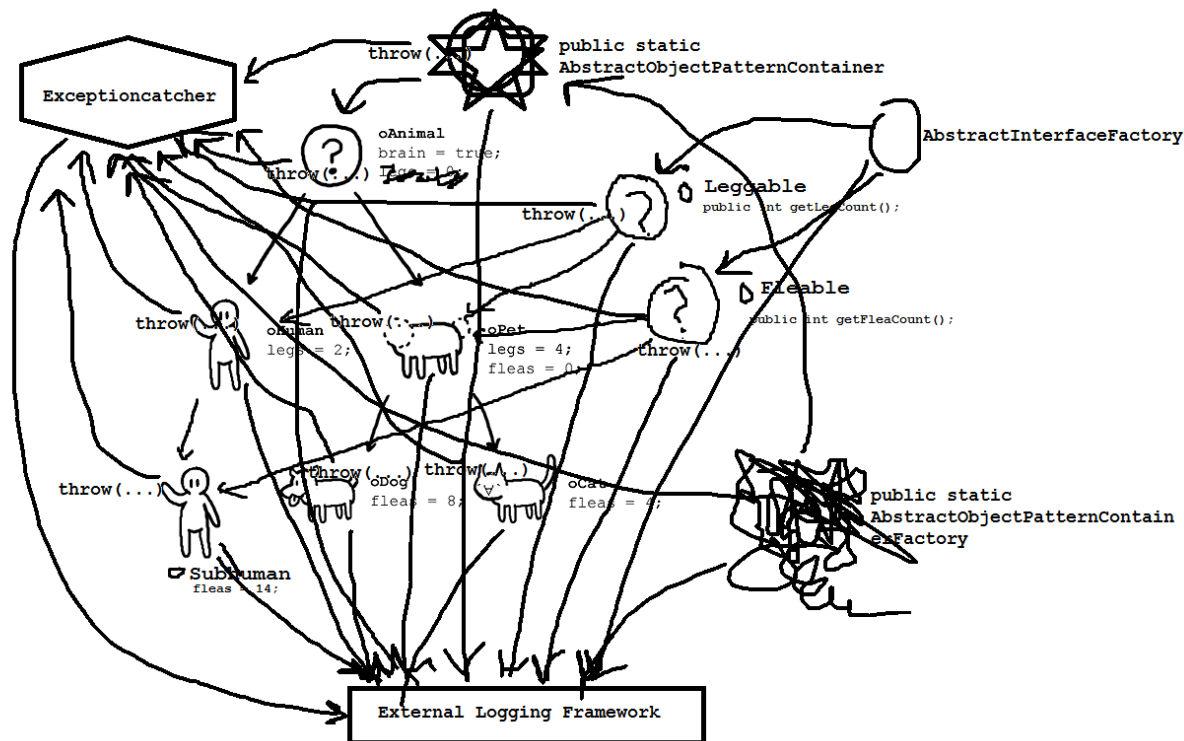


Source: <https://www.slavoslaw.pl/pogrzeb-po-slowiansku-palenie-i-grzebanie/>

What OOP users claim



What actually happens



**WHAT'S THE OBJECT-ORIENTED
WAY TO BECOME WEALTHY**

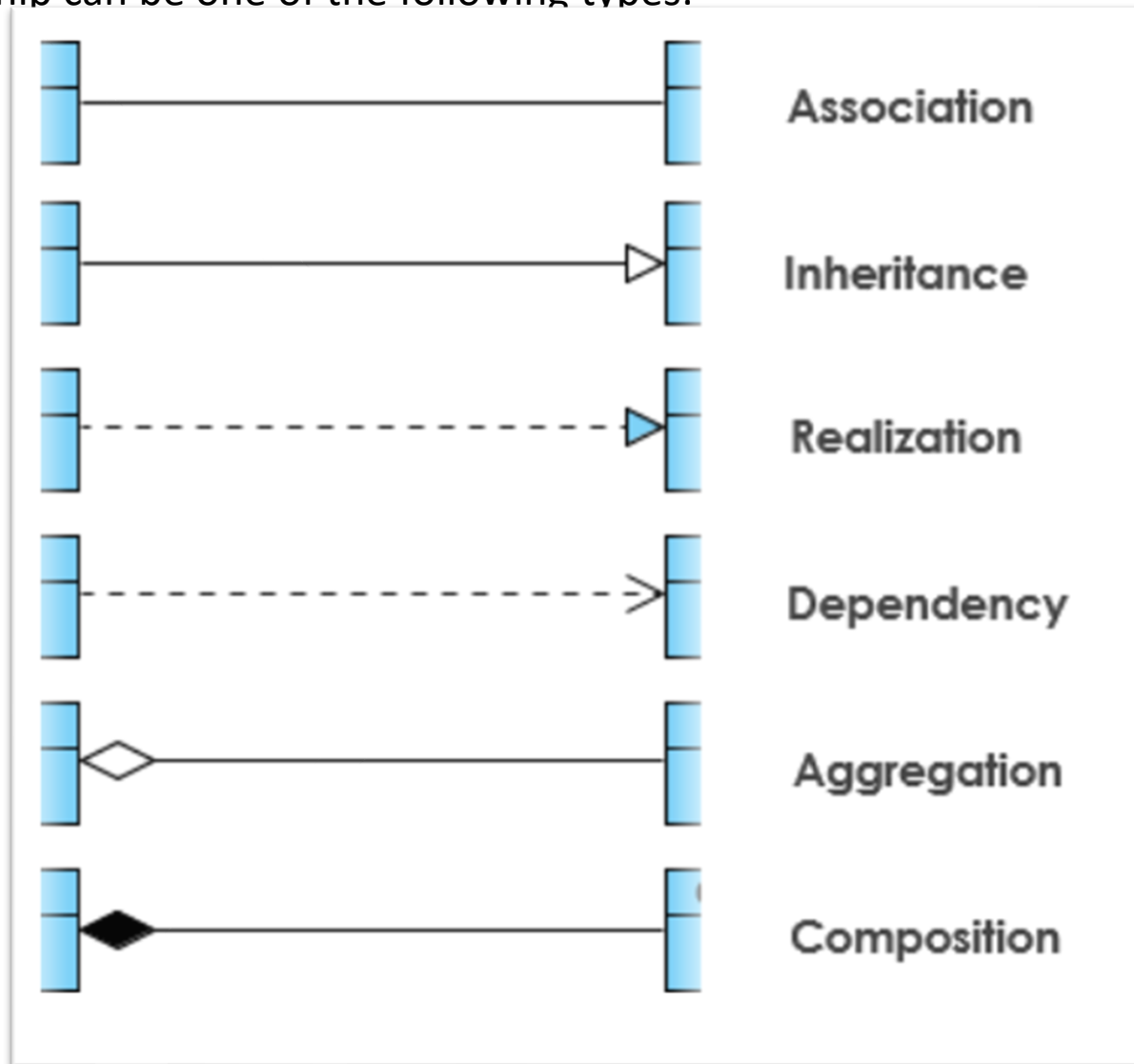


INHERITANCE

Source: <https://codeburst.io/youre-7-mins-away-from-swift-4-e42e003d6f69?gi=2192b3f4a5b>

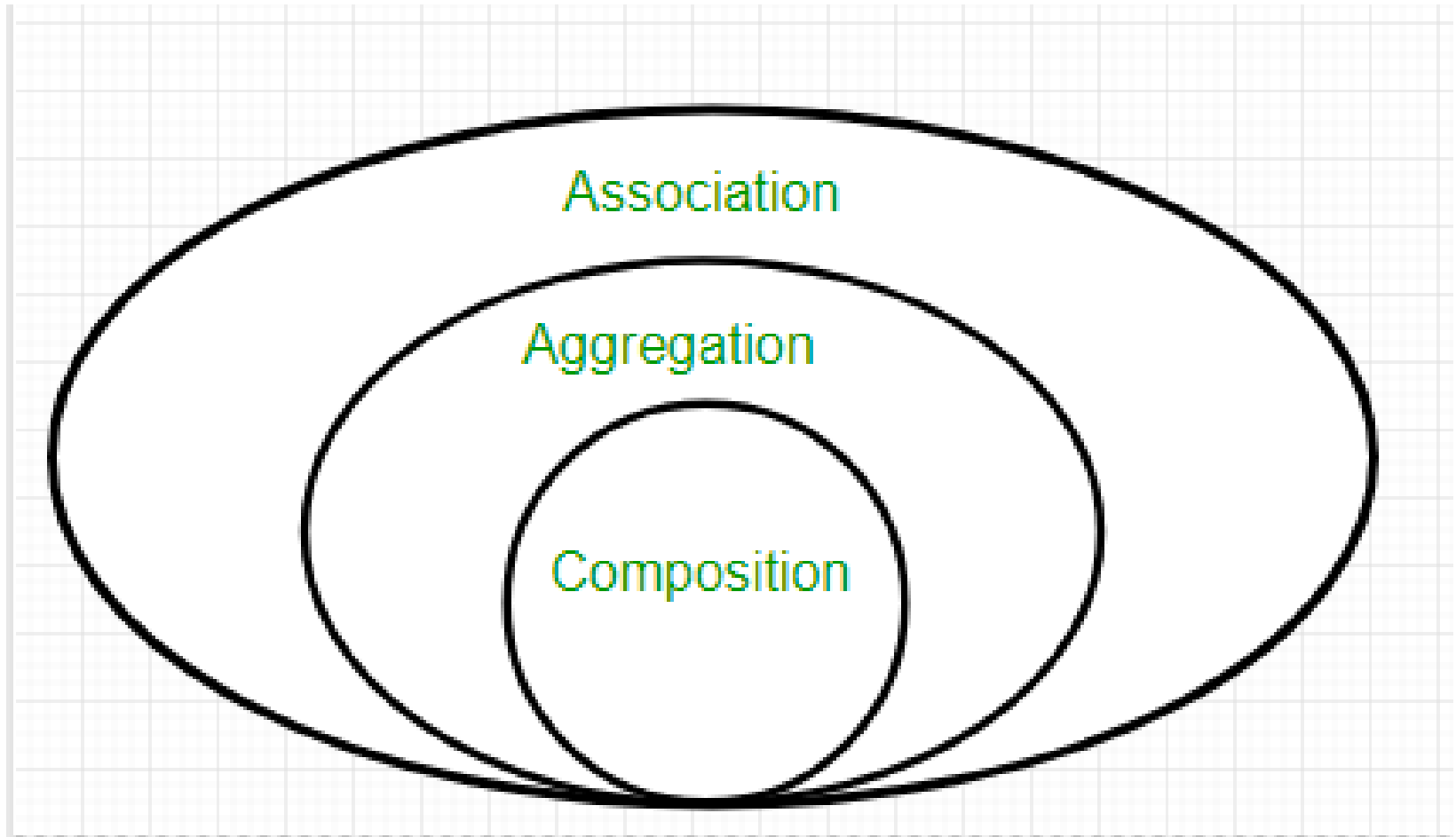
UML CLASS NOTATION

A relationship can be one of the following types:



Source: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

Composition and **Aggregation** are the two forms of association.

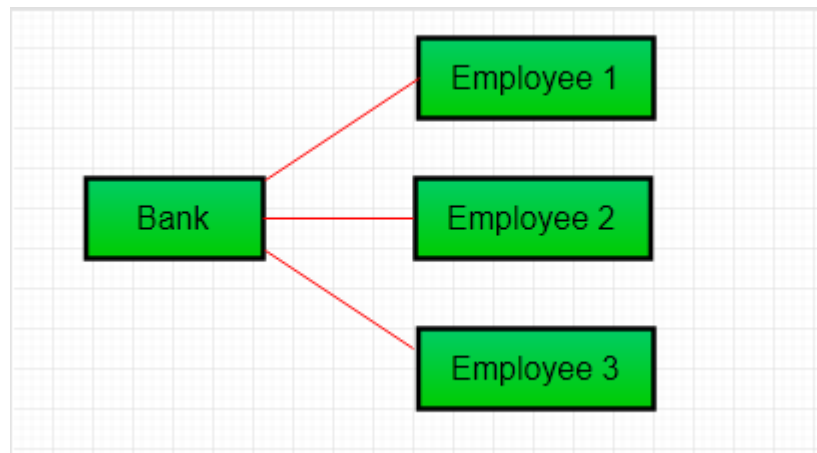


Source: <https://www.geeksforgeeks.org/association-composition-aggregation-java/>

ASSOCIATION

```
// Java program to illustrate the  
// concept of Association  
import java.io.*;
```

```
// class bank  
class Bank  
{  
    private String name;  
  
    // bank name  
    Bank(String name)  
    {  
        this.name = name;  
    }  
  
    public String getBankName()  
    {  
        return this.name;  
    }  
}
```



ASSOCIATION

// Java program to illustrate the
// concept of Association

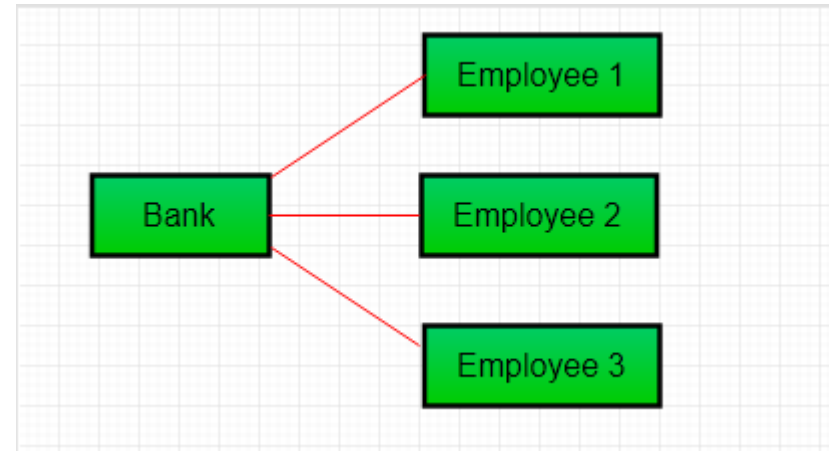
// employee class

```
class Employee
{
    private String name;

    // employee name
    Employee(String name)
    {
        this.name = name;
    }

    public String getEmployeeName()
    {
        return this.name;
    }
}

//
```

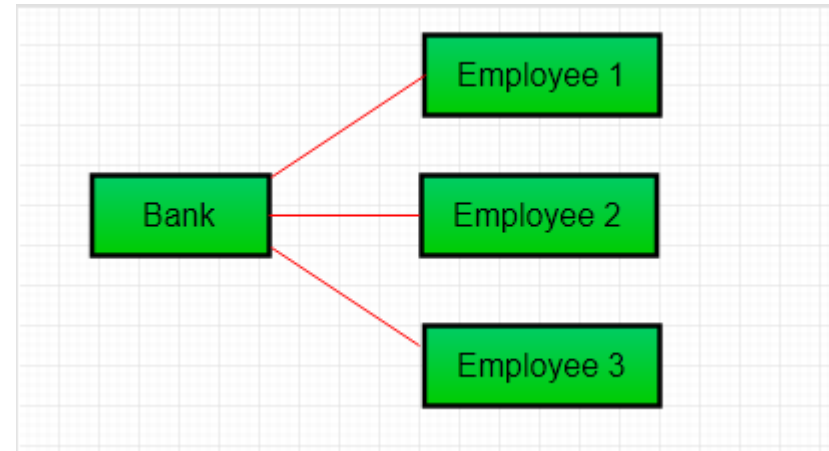


ASSOCIATION

```
// Java program to illustrate the  
// concept of Association
```

```
// Association between both the  
// classes in main method  
class Association
```

```
{  
    public static void main (String[] args)  
    {  
        Bank bank = new Bank("Axis");  
        Employee emp = new Employee("Neha");  
  
        System.out.println(emp.getEmployeeName() +  
            " is employee of " + bank.getBankName());  
    }  
}
```

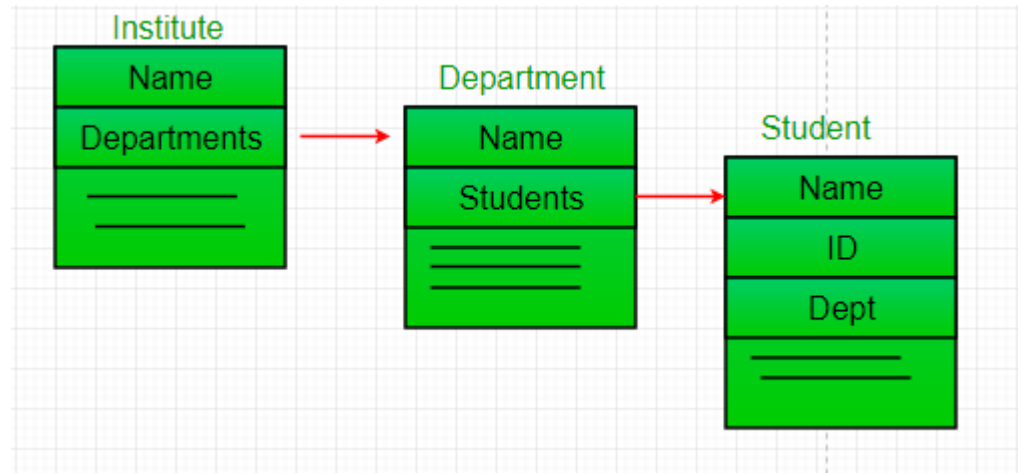


AGGREGATION

```
// Java program to illustrate
//the concept of Aggregation.
import java.io.*;
import java.util.*;
```

```
// student class
class Student
{
    String name;
    int id ;
    String dept;

    Student(String name, int id, String dept)
    {
        this.name = name;
        this.id = id;
        this.dept = dept;
    }
}
```



AGGREGATION

/* Department class contains list of student Objects. It is associated with student class through its Object(s). */

class Department

{

String name;

private List<Student> students;

Department(String name, List<Student> students)

{

 this.name = name;

 this.students = students;

}

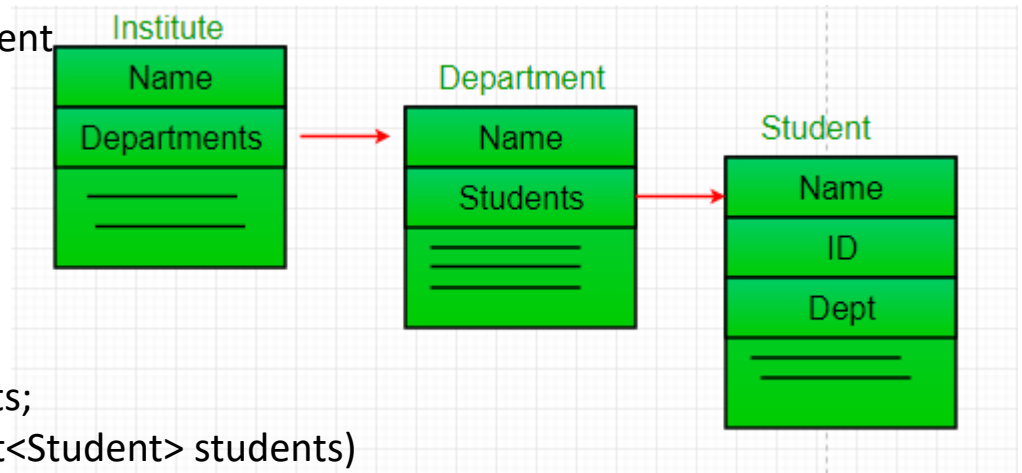
public List<Student> getStudents()

{

 return students;

}

}



AGGREGATION

/* Institute class contains list of Department Objects. It is associated with Department class through its Object(s).*/

class Institute

{

```
String instituteName;  
private List<Department> departments;
```

```
Institute(String instituteName, List<Department> departments)
```

```
{
```

```
    this.instituteName = instituteName;  
    this.departments = departments;
```

```
}
```

```
// count total students of all departments
```

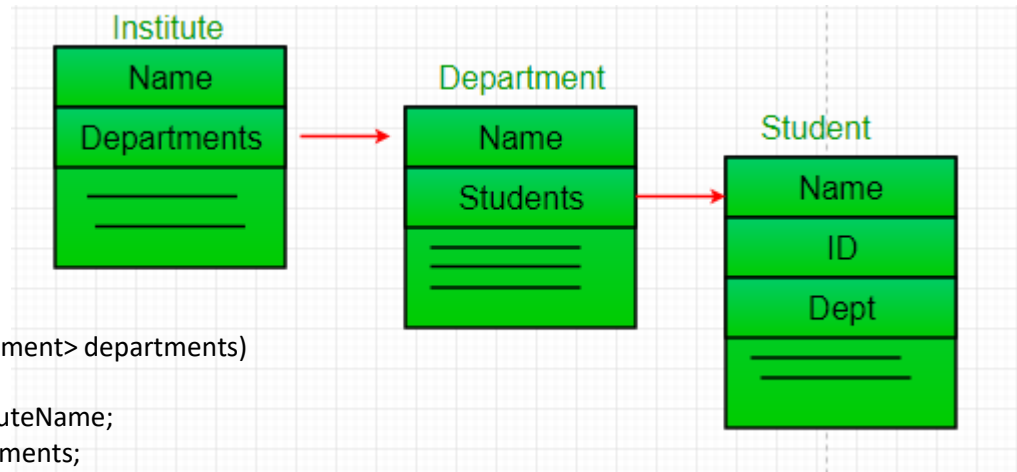
```
// in a given institute
```

```
public int getTotalStudentsInInstitute()
```

```
{
```

```
    int noOfStudents = 0;  
    List<Student> students;  
    for(Department dept : departments)  
    {  
        students = dept.getStudents();  
        for(Student s : students)  
        {  
            noOfStudents++;  
        }  
    }  
    return noOfStudents;
```

```
}
```



Source: <https://www.geeksforgeeks.org/association-composition-aggregation-java/>

AGGREGATION

```
// main method  
class GFG  
{
```

```
    public static void main (String[] args)  
    {
```

```
        Student s1 = new Student("Mia",  
        Student s2 = new Student("Priya"  
        Student s3 = new Student("John"  
        Student s4 = new Student("Rahul
```

```
        // making a List of  
        // CSE Students.  
        List <Student> cse_students = new  
        cse_students.add(s1);  
        cse_students.add(s2);
```

```
        // making a List of  
        // EE Students  
        List <Student> ee_students = new ArrayList<Student>();  
        ee_students.add(s3);  
        ee_students.add(s4);
```

```
        Department CSE = new Department("CSE", cse_students);  
        Department EE = new Department("EE", ee_students);
```

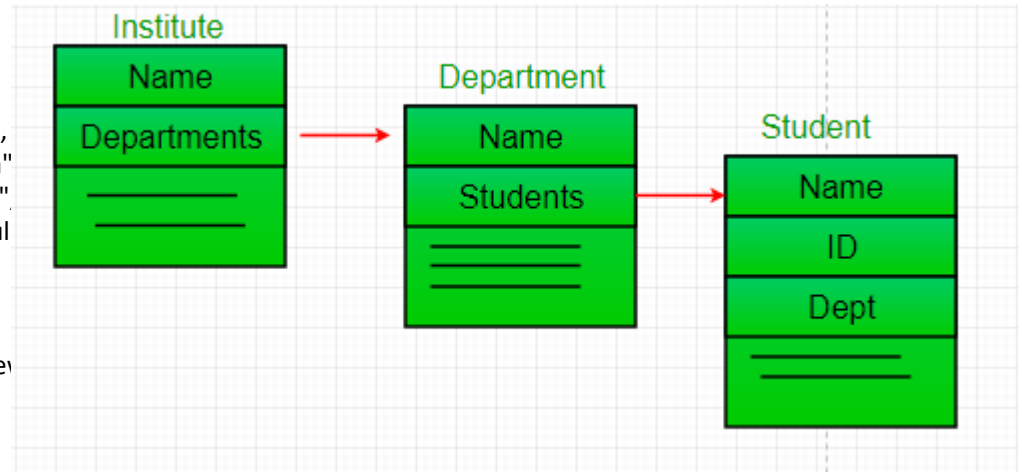
```
        List <Department> departments = new ArrayList<Department>();  
        departments.add(CSE);  
        departments.add(EE);
```

```
        // creating an instance of Institute.  
        Institute institute = new Institute("BITS", departments);
```

```
        System.out.print("Total students in institute: ");  
        System.out.print(institute.getTotalStudentsInInstitute());
```

```
    }
```

```
}
```



Source: <https://www.geeksforgeeks.org/association-composition-aggregation-java/>

UML CLASS NOTATION

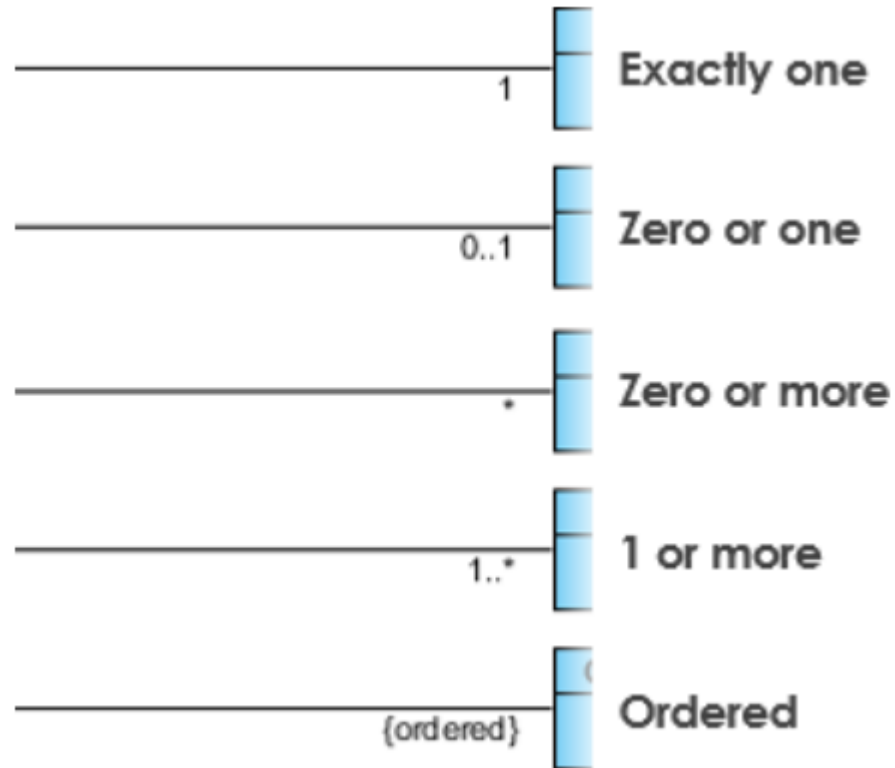
Composition : Since Engine is-part-of Car, the relationship between them is Composition. Here is how they are implemented between Java classes.

```
public class Car {  
    //final will make sure engine is initialized  
    private final Engine engine;  
  
    public Car(){  
        engine = new Engine();  
    }  
}  
  
class Engine {  
    private String type;  
}
```

UML CLASS NOTATION

Cardinality is expressed in terms of:

- one to one
- one to many
- many to many



Source: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>



Source: <https://www.improgrammer.net/java-programming-jokes-best-programmer-humor/>

TRY, CATCH

```
class Example1 {
    public static void main(String args[]) {
        int num1, num2;
        try {
            /* We suspect that this block of statement can throw
             * exception so we handled it by placing these statements
             * inside try and handled the exception in catch block
             */
            num1 = 0;
            num2 = 62 / num1;
            System.out.println(num2);
            System.out.println("Hey I'm at the end of try block");
        }
        catch (ArithmeticException e) {
            /* This block will only execute if any Arithmetic exception
             * occurs in try block
             */
            System.out.println("You should not divide a number by zero");
        }
        catch (Exception e) {
            /* This is a generic Exception handler which means it can handle
             * all the exceptions. This will execute if the exception is not
             * handled by previous catch blocks.
             */
            System.out.println("Exception occurred");
        }
        System.out.println("I'm out of try-catch block in Java.");
    }
}
```

Output:

```
You should not divide a number by zero
I'm out of try-catch block in Java.
```

Source: <https://beginnersbook.com/2013/04/try-catch-in-java/>

TRY, CATCH

Before you can catch an exception, some code somewhere must throw one.

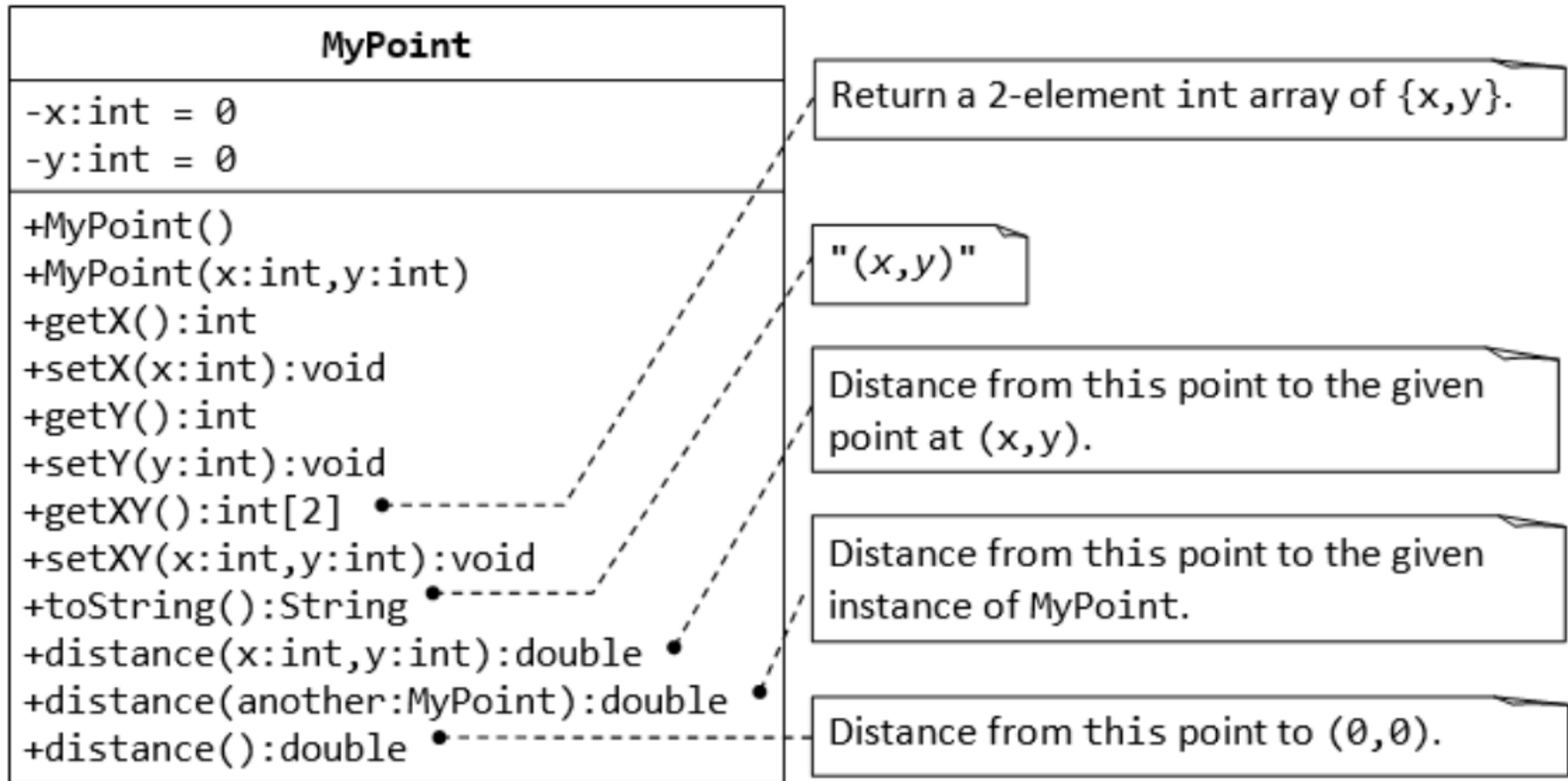
```
public Object pop() {  
    Object obj;  
  
    if (size == 0) {  
        throw new EmptyStackException();  
    }  
  
    obj = objectAt(size - 1);  
    setObjectAt(size - 1, null);  
    size--;  
    return obj;  
}
```

Any code can throw an exception: your code, code from a package written by someone else such as the packages that come with the Java platform, or the Java runtime environment. Regardless of what throws the exception, it's always thrown with the throw statement.

EXERCISES 05_01

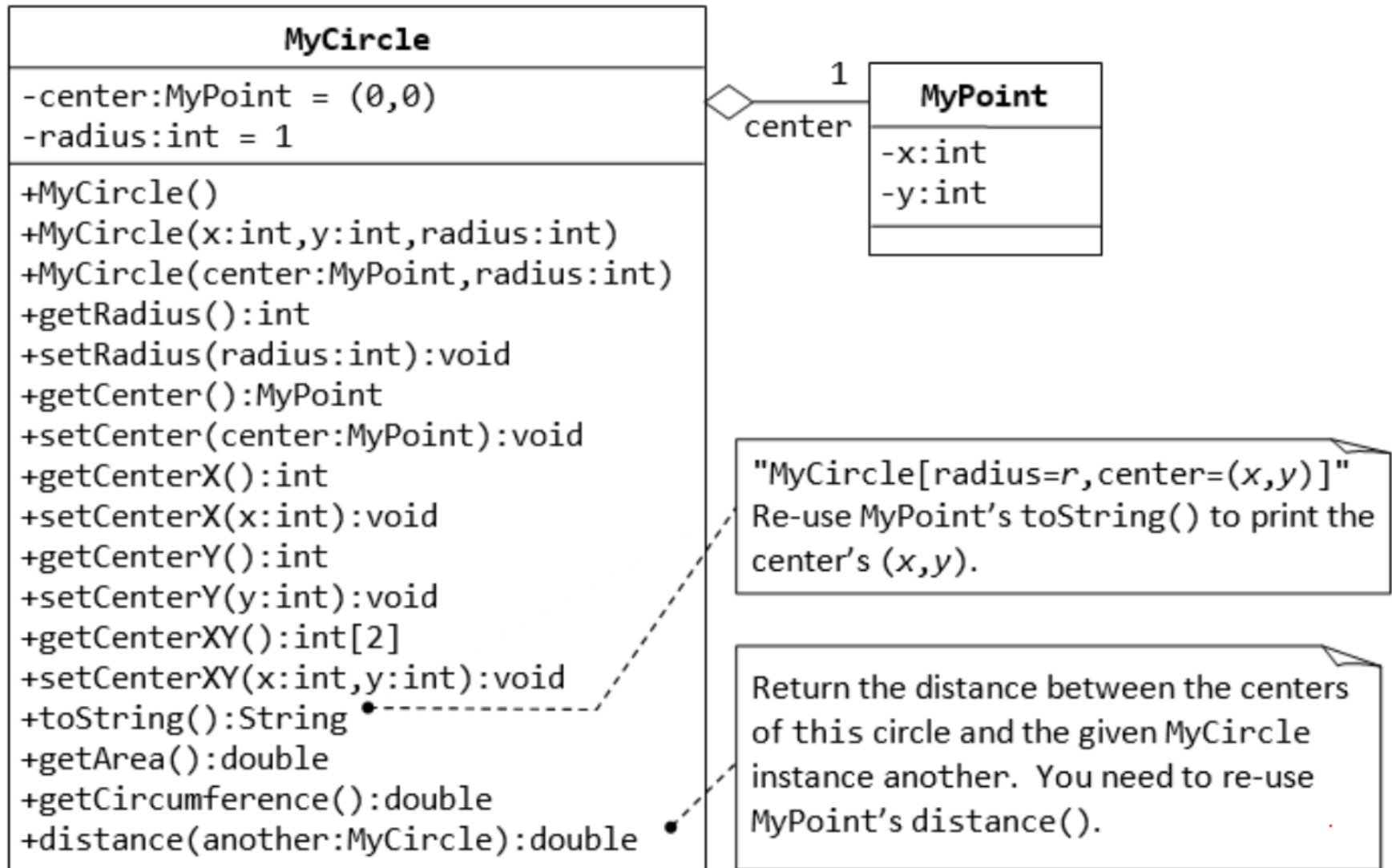
Implement the following UML chart

Write a program that allocates 10 points in an array of MyPoint, and initializes to (1, 1), (2, 2), ... (10, 10).



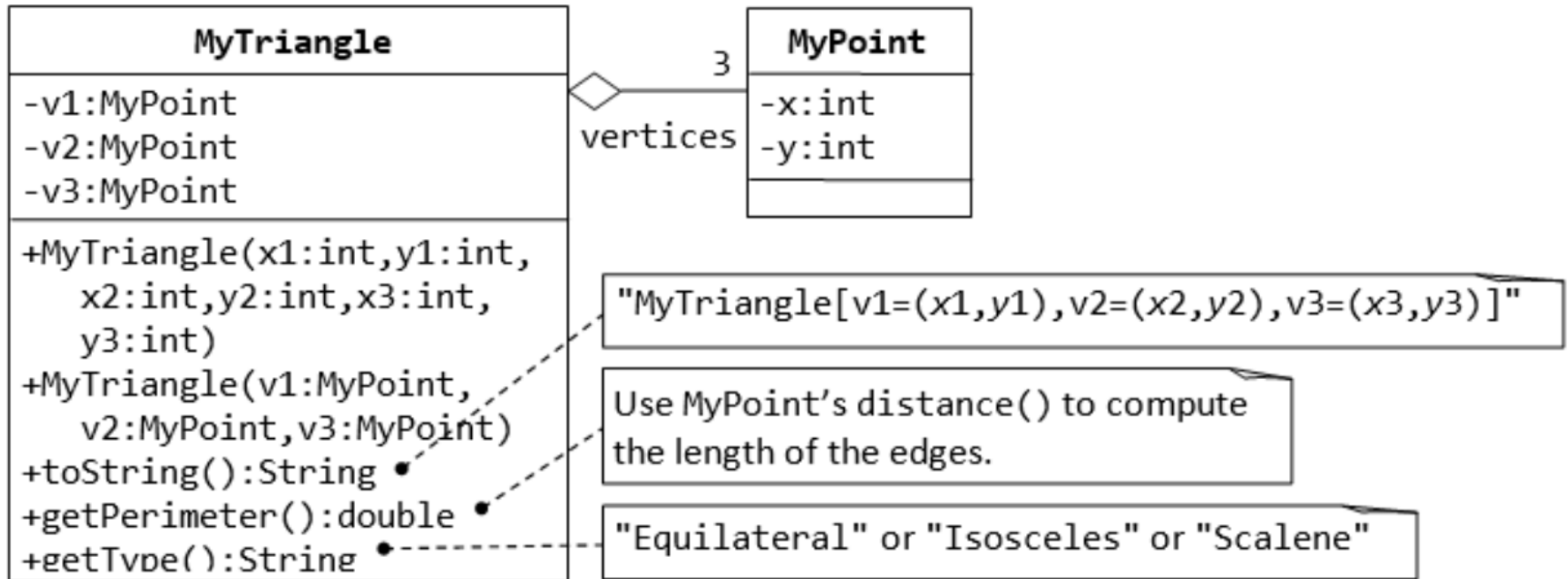
EXERCISES 05_02

Implement the following UML chart



EXERCISES 05_03

Implement the following UML chart

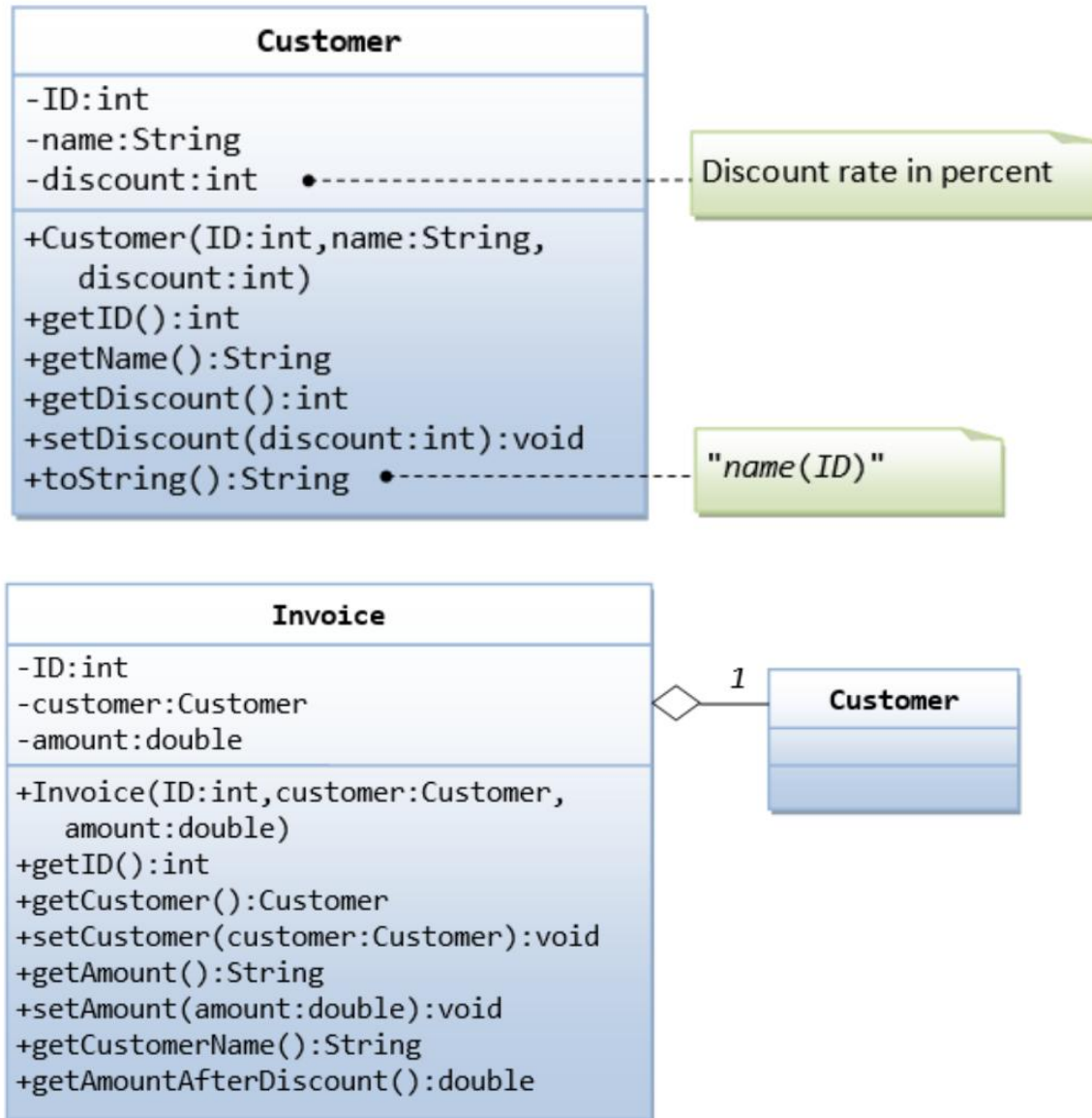


EXERCISES 05_04

Design a `MyRectangle` class which is composed of two `MyPoint` instances as its top-left and bottom-right corners. Draw the class diagrams, write the codes, and write the test drivers.

EXERCISES 05_05

Implement the following UML chart



EXERCISES 05_06 DESIGN CHEMICAL ELEMENTS

The periodic table of chemical elements classifies and displays all **chemical elements**. Each chemical element has a unique symbolic name and atomic number (number of protons). Chemical elements are grouped together by common characteristics (alkali metal, poor metal, ...) called the chemical series.

Examples of chemical elements:

H (hydrogenium): Hydrogen with atomic number 1.

O (oxygenium): Oxygen with atomic number 8.

K: Potassium with atomic number 19. It is an alkali metal.

Zn: Zinc with atomic number 30. It is a transition metal.

Ga: Gallium with atomic number 31. It is a metal.

We consider the following chemical series:

Alkali metals are all chemical element with atomic number 3, 11, 19, 37, 55, or 87

Transition metals are all chemical elements with atomic number from 21 to 31, 39 to 48, 72 to 80, and 104 to 112.

Metals are all chemical elements with atomic number 13, 49, 50, 81, 82, 83, 113, 114, 115, or 116.

Design and implement a class `ChemicalElement`. The class should contain methods to retrieve for a chemical element its name, symbolic name, atomic number, and which type of metal it is (three different methods for each metal property).

EXERCISES 05_07 STATISTIC

Write a program called GradesStatistics, which reads in n grades (of int between 0 and 100, inclusive) and displays the average, minimum, maximum, median and standard deviation. Display the floating-point values up-to 2 decimal places. Your output shall look like:

```
Enter the number of students: 4
Enter the grade for student 1: 50
Enter the grade for student 2: 51
Enter the grade for student 3: 56
Enter the grade for student 4: 53
The grades are: [50, 51, 56, 53]
The average is: 52.50
The median is: 52.00
The minimum is: 50
The maximum is: 56
The standard deviation is: 2.29
```

The formula for calculating standard deviation is:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} x_i^2 - \mu^2}, \text{ where } \mu \text{ is the mean}$$

EXERCISES 05_08, 05_09

Rozwiąż dwa zadania z serwisu codinggames.com
Wkomituj opis i rozwiązanie do swojego repozytorium.

EXERCISES (BONUS)

Write algorithm which solve Sudoku puzzle. Test it on with the following example:

```
*****
*5  ?  ?*?  8  ?*?  4  9*
*?  ?  ?*5  ?  ?*?  3  ?*
*?  6  7*3  ?  ?*?  ?  1*
*****
*1  5  ?*?  ?  ?*?  ?  ?*
*?  ?  ?*2  ?  8*?  ?  ?*
*?  ?  ?*?  ?  ?*?  1  8*
*****
*7  ?  ?*?  ?  4*1  5  ?*
*?  3  ?*?  ?  2*?  ?  ?*
*4  9  ?*?  5  ?*?  ?  3*
*****
```